Written Assignment

DLMAIRIL01 – Reinforcement Learning

IU International University of Applied Sciences

Degree programme: [Full official programme name]

# Learning and Search in AlphaGo and AlphaGo Zero: An Evidence-Weighted Comparison

[Full name]

Matriculation number: [ ]

Supervisor/Tutor: [Title and name]

Submission date: [ ]

# Contents

**List of abbreviations** ..... **III**
**1 Introduction** ..... **1**
**2 Foundations** ..... **2**
2.1 Go as a two-player zero-sum game of perfect information ..... 2
2.2 Monte Carlo Tree Search and bandit-based selection ..... 3
**3 AlphaGo: learned priors and evaluations inside MCTS** ..... **4**
3.1 Network inventory and training pipeline ..... 4
3.2 From imitation to reinforcement learning ..... 5
3.3 Value network and leaf evaluation ..... 6
3.4 Integration into search and component contribution ..... 7
**4 AlphaGo Zero: search as a policy-improvement operator** ..... **8**
4.1 Dual-head residual network and minimal input ..... 8
4.2 Self-play loop and training targets ..... 8
4.3 Search without rollouts and exploration mechanisms ..... 9
**5 Comparison and critical evaluation** ..... **10**
5.1 From Fan via Lee and Master to Zero ..... 10
5.2 Attributing the performance gain ..... 12
5.3 Limits: human knowledge, compute, robustness, reproducibility and transfer ..... 13
**6 Conclusion** ..... **14**
**References** ..... **16**

# List of abbreviations

| | |
|---|---|
| AI | Artificial intelligence |
| CPU | Central processing unit |
| DNN | Deep neural network |
| GPU | Graphics processing unit |
| ICLR | International Conference on Learning Representations |
| KGS | KGS Go Server |
| MCTS | Monte Carlo Tree Search |
| NeurIPS | Neural Information Processing Systems |
| PMLR | Proceedings of Machine Learning Research |
| PUCT | Predictor-guided UCT (AlphaGo's variant) |
| REINFORCE | REward Increment = Nonnegative Factor × Offset Reinforcement × Characteristic Eligibility |
| RL | Reinforcement learning |
| SL | Supervised learning |
| SQ | Sub-question |
| TPU | Tensor Processing Unit |
| UCB1 | Upper Confidence Bound algorithm 1 |
| UCL | University College London |
| UCT | Upper Confidence bounds applied to Trees |

# 1 Introduction

Go made the limitations of brute-force artificial intelligence (AI) particularly visible. Its approximate branching factor of 250 and typical depth of 150 moves make exhaustive enumeration impractical, while evaluating an unfinished position is itself difficult (Silver et al., 2016, p. 484). Monte Carlo Tree Search (MCTS) had already transformed computer Go: Gelly et al. (2012, section "Introduction", Figure 1) describe substantial progress, including successes against professionals under restricted conditions. Nevertheless, strong amateur performance on the full board remained the relevant pre-AlphaGo baseline (Silver et al., 2016, p. 484). The breakthrough was consequently a change in competitive capability, rather than the invention of computer Go.

AlphaGo Fan defeated Fan Hui 5–0 without handicap in October 2015; AlphaGo Lee subsequently defeated Lee Sedol 4–1, and AlphaGo Master achieved 60–0 against top professionals online (Silver et al., 2016, p. 488; Silver et al., 2017, pp. 21–22). The original study described defeating a professional on the full board as a feat previously expected to remain a decade away (Silver et al., 2016, p. 488). This substantiates the contemporary surprise, while distinguishing professional-level success from the later, stronger evidence of superhuman performance. The open analytical question is which interacting changes explain that progression.[1]

The main research question is: How do deep neural networks and MCTS interact in AlphaGo, and to what extent can AlphaGo Zero's performance gain be attributed to search-generated training targets rather than changes in architecture and input representation? Three sub-questions organise the answer: which networks perform which functions (SQ1, Chapter 3); how their integration into MCTS affects performance (SQ2, Section 3.4); and which changes in Zero explain measurable gains (SQ3, Chapters 4–5). The working thesis is joint: search-generated targets change the learning process, whereas architectural ablations establish substantial additional gains (Silver et al., 2017, pp. 8–9).

A focused literature review and source-verification search was conducted on 5 November 2026, seeded by the draft's references and assessment's recommendations. The course book supplied orientation and bibliographic leads. Web searches used specific titles, including "Multi-armed bandits with episode context", "Bandit based Monte-Carlo planning" and "Policy improvement by planning with Gumbel". Sources were retrieved through Nature/UCL Discovery, Springer, arXiv, NeurIPS,

1 Locators refer to the versions consulted: printed journal/proceedings pages where present; numbered accepted-manuscript pages for Silver et al. (2017, 2018) and Schrittwieser et al. (2020); PDF pages for Anthony et al. (2017) and Marcus (2018); manuscript section labels for Coulom (2007) and Gelly et al. (2012). Rosin is the undated author manuscript, not the inaccessible 2011 journal PDF. [VERIFY: confirm the date/version of Rosin's author manuscript before replacing n.d. with a year.]

PMLR and ICLR. Selection favoured peer-reviewed system reports, foundational methods, independent reimplementation and critical evaluation; an external preprint supplies an interpretative position. Comparison dimensions are data, objectives, architecture, search, compute and evaluation conditions. This purposive procedure supports technical attribution rather than systematic coverage.

AlphaGo here denotes Fan unless Lee or Master is named; Zero denotes the 2017 self-play system. Later AlphaZero and MuZero test claims about design dependence and transfer. The analysis proceeds from game and search foundations to the original network pipeline, Zero's feedback loop, quantitative attribution and external limitations. Evidence is confined to published measurements; experimental replication lies beyond this review (Silver et al., 2016, pp. 484–488; Silver et al., 2017, pp. 3–12).

# 2 Foundations

The distinction between learning a reusable decision function and searching a particular position underpins the comparison. The following sections establish the game formulation and the allocation of a finite search budget.

## 2.1 Go as a two-player zero-sum game of perfect information

Go is naturally formulated as an alternating, two-player Markov game with deterministic transitions and perfect information. The state includes the player to move and rule-relevant history; legal placements and pass are actions. An exact simulator implements transitions and terminal scoring. The players' objectives are opposed, with outcomes +1 and −1 from the respective player's perspective. Perfect information describes the game, whereas a chosen neural input can still omit information relevant to particular rules (Silver et al., 2016, Methods section "Problem setting"; Silver et al., 2017, pp. 20, 27).

A policy p(a|s) distributes probability across legal actions. Its value function estimates the expected return obtained from a state under that policy, rather than mechanically identifying the best move (Sutton & Barto, 2018, pp. 58–59). AlphaGo's signed outcome convention means that a value near zero represents approximately balanced expected outcomes under the represented play. A win for Black receives opposite signs when evaluated from Black's and White's perspectives. Tree backups must preserve that convention at successive alternating nodes (Silver et al., 2016, p. 486, Methods section "Search algorithm").

The simulator supplies the consequences of moves; neural networks estimate their strategic relevance. This separates knowledge of transition dynamics from competence in choosing actions. Even exact rules leave an enormous decision problem, so learned priors restrict effective breadth and learned values truncate effective depth (Silver et al., 2016, p. 484). The resulting system combines an available model with function approximation. Its central resource question is how much additional

evidence search can gather beyond the predictions already amortised into network parameters (Sutton & Barto, 2018, pp. 159–161, 185–188).

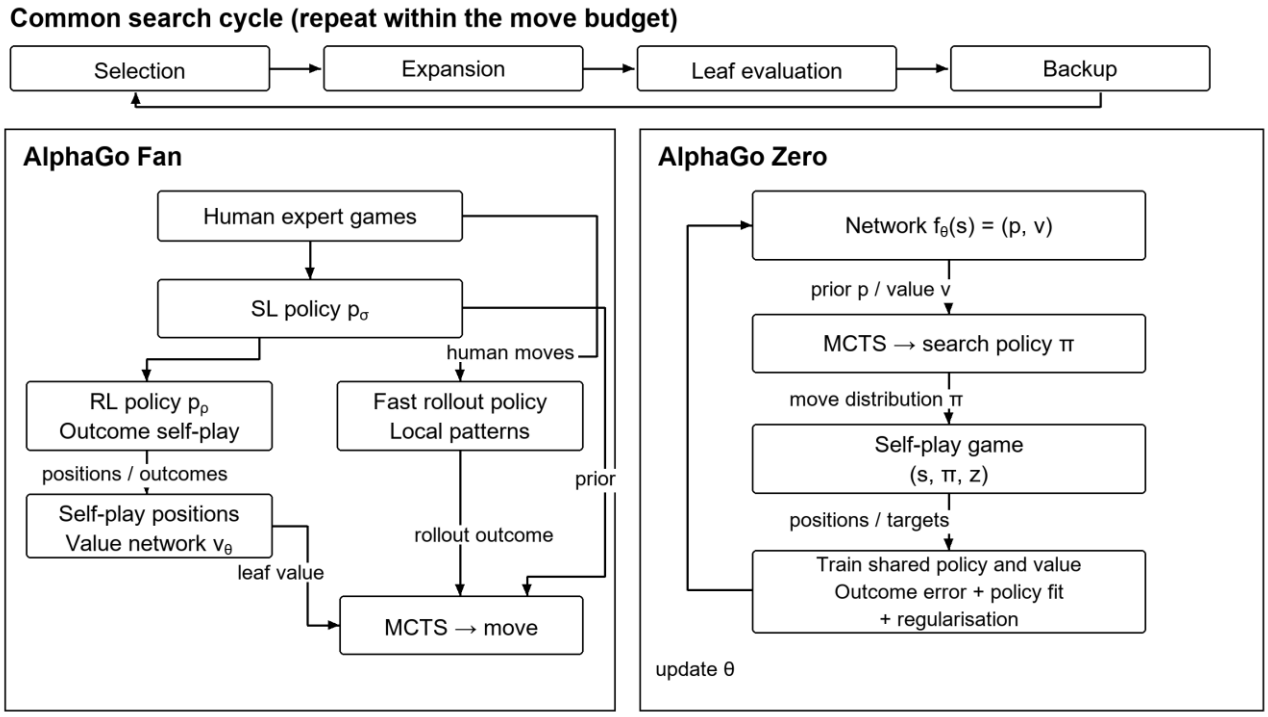
## 2.2 Monte Carlo Tree Search and bandit-based selection

MCTS incrementally allocates computation to a partial tree. Coulom (2007, section "Algorithm Structure") develops a framework combining selective tree growth with Monte Carlo evaluation; Browne et al. (2012, pp. 5–6) consolidate its four stages. Selection follows existing edges, expansion introduces a new node, simulation evaluates its continuation, and backup updates statistics along the visited path. In neural-guided variants, evaluation can replace a terminal rollout. Figure 1 retains this common cycle while separating Fan's training pipeline from Zero's closed learning loop.

Selection addresses an exploration–exploitation problem. UCB1, the upper-confidence-bound algorithm, combines empirical rewards with a confidence bonus for less-sampled actions under bounded, stationary bandit assumptions (Auer et al., 2002, pp. 236–237). Upper Confidence bounds applied to Trees (UCT) extends bandit allocation to lookahead planning, with consistency results under the analysed conditions (Kocsis & Szepesvári, 2006, pp. 283–286). Prior-guided selection adds a predictor's recommendations, drawing on Rosin's framework (Rosin, n.d., pp. 1–2). AlphaGo adapts this idea into its PUCT selection rule, examined in Section 4.3 (Silver et al., 2016, Methods section "Search algorithm").

Learning and planning operate at different scales. The neural network generalises experience across positions, whereas search concentrates further computation on the current decision. Sutton and Barto's (2018, pp. 159–163, 185–188) integration of planning and learning, including Dyna, provides the broader methodological context. Neural backpropagation updates parameters; tree backup updates local counts and values. Accordingly, an inference-time search can revise a move recommendation while its network remains fixed. AlphaGo instantiates this division of labour with learned move guidance, learned outcome evaluation and a fast rollout policy (Silver et al., 2016, pp. 484–486).

**Fig. 1 Learning pipelines and the common search cycle**




Source: Own representation based on Browne et al. (2012, pp. 5–6), Silver et al. (2016, pp. 484–486) and Silver et al. (2017, pp. 3–6). Diagram labels and code prepared with GitHub Copilot (GitHub, 2026); prompt excerpt: "Fig. 1, built as a real Word figure from the mermaid blueprint." The supplied blueprint determined the structure. Arrows denote data or evaluation flow.

# 3 AlphaGo: learned priors and evaluations inside MCTS

## 3.1 Network inventory and training pipeline

AlphaGo Fan assigns distinct objectives to three deep models and one lightweight policy. The supervised-learning (SL) policy predicts expert actions; its reinforcement-learning (RL) successor optimises self-play outcomes; the value network predicts outcomes under the RL policy. The fast rollout policy instead uses a linear softmax over local patterns. Table 1 makes the network-to-task correspondence explicit and includes Zero's later replacement for comparison (Silver et al., 2016, pp. 484–486, Methods section "Neural network architecture"; Silver et al., 2017, pp. 27–29).

**Tab. 1 Network inventory and operational roles**

| Model | Input | Architecture | Training data | Objective | Role |
|---|---|---|---|---|---|
| Fan SL policy | 48 board/tactical planes | 13-layer convolutional network; softmax | 30 M KGS positions | Expert-move likelihood | MCTS priors |
| Fan RL policy | Same as SL | Same network; SL initialisation | Self-play vs earlier policies | Expected signed outcome | Generates value-training games |
| Fan value | 48 planes + colour | Convolutional trunk; 256-unit dense layer; tanh | 30 M positions, separate games | Squared outcome error | Leaf value estimate |
| Fan rollout | Small local patterns | Linear softmax; not a DNN | Human expert moves | Move likelihood | Fast terminal rollouts |
| Zero policy/value | 17 history/colour planes | 20/40-block residual tower; two heads | Search-guided self-play | Policy cross-entropy + value error + regularisation | Priors, leaf values; fitted to (s, π, z) |

Source: Own representation based on Silver et al. (2016, pp. 484–486, Methods sections "Features for policy/value network" and "Neural network architecture") and Silver et al. (2017, pp. 6, 27–29).

The original policy input has 48 feature planes on a 19 × 19 grid, encoding stones, liberties, capture and legality information, move history and ladder-related tactical information. The value input adds colour to play. Convolution shares spatial filters across board locations, allowing recurring local configurations to contribute to a reusable position representation. The match policy has 13 convolutional layers, with 192 filters in the hidden layers; the final softmax produces move probabilities (Silver et al., 2016, Methods sections "Features for policy/value network" and "Neural network architecture").

Training starts with human demonstrations rather than unstructured exploration. The 30 million positions from the KGS Go Server provide state–move pairs, and the trained SL policy achieves 57.0% held-out expert-move accuracy with all features. The rollout policy achieves 24.2% accuracy but selects an action in 2 microseconds, compared with 3 milliseconds for the deep policy. That latency difference explains why the weaker model remains useful for long continuations: replacing it with a stronger but slower policy changes the search budget available per decision (Silver et al., 2016, p. 485).

## 3.2 From imitation to reinforcement learning

For a demonstrated action a* in state s, the negative log-likelihood loss in Equation 1 is an explanatory single-example form of the original supervised objective. Minimisation increases the probability assigned to that expert action. The target establishes agreement with human play; it supplies dense, useful feedback before the system has learned to generate strong games for itself (Silver et al., 2016, pp. 484–485).

$$L_{SL} = -log\, p_\sigma(a*|s) \quad (1)$$

The RL policy starts from the SL parameters but replaces demonstration matching with outcome-weighted policy-gradient updates. Equation 2 expresses AlphaGo's update, where ρ denotes policy parameters and $z_t$ the final result from the player-to-move perspective. Positive outcomes reinforce sampled actions; negative outcomes reverse the direction. Williams (1992, pp. 233–234) establishes the likelihood-gradient principle of REINFORCE, and the episodic application is described by Silver et al. (2016, p. 485). A single game is a noisy trajectory-level signal; improvement arises across repeated experience.

$$\Delta\rho \propto z_t \nabla_\rho \log p_\rho(a_t|s_t) \quad (2)$$

The reported effect is substantial. Without search, the RL policy wins more than 80% against its SL initialisation and 85% against Pachi, which uses 100,000 simulations per move. During training, opponents are randomly sampled from earlier policy versions to stabilise the changing learning problem (Silver et al., 2016, p. 485). This evidence links outcome-based learning to improved direct play, rather than assuming that a new objective automatically improves performance.

Inside the complete search, however, the SL prior performs better than the stronger RL policy. The authors suggest that its broader distribution of promising human moves helps search; this proposed explanation is less firmly established than the measured direction of the comparison. No separate effect size for the prior swap is stated in that passage. The RL policy instead supplies the behaviour whose outcomes train the value network, making the next stage a distinct contribution to the final agent (Silver et al., 2016, pp. 486–487).

## 3.3 Value network and leaf evaluation

The value network approximates $v^{p_\rho}$(s), the expected signed outcome under the RL policy playing both sides, rather than a solved minimax value. Its tanh output lies between −1 and +1. Equation 3 gives the single-example squared-error objective used to fit completed self-play outcomes. The policy therefore tells search where to look, while the value function estimates the consequences of stopping a continuation early (Silver et al., 2016, p. 486).

$$L_{value} = (z - v_\theta(s))^2 \quad (3)$$

Data construction is crucial to this evaluation function. Training on whole human games produced mean squared errors of 0.19 on training data and 0.37 on test data, reflecting correlated positions sharing one outcome. The replacement dataset used 30 million distinct positions, one per self-play game, and yielded 0.226 and 0.234 respectively. The smaller gap supports reduced overfitting, although simultaneous changes in sampling and data-generating policy limit causal attribution (Silver et al., 2016, p. 486).

At a leaf, AlphaGo combines the learned estimate with the terminal result of a fast rollout as in Equation 4. Here $s_L$ denotes the leaf state, $z_L$ the rollout outcome and λ its mixing weight. These

estimators summarise play by different policies, so their combination offers complementary evidence, rather than two interchangeable measurements. The network alone approaches the accuracy of strong-policy rollouts with 15,000 times less computation in the reported evaluation. The resulting computational advantage makes learned evaluation usable repeatedly within search (Silver et al., 2016, p. 486).

$$V(s_L) = (1 - \lambda)v_\theta(s_L) + \lambda z_L \quad (4)$$

## 3.4 Integration into search and component contribution

Each search edge stores a prior, a visit count and an action-value estimate. Selection trades off estimated value against prior-weighted exploration; expansion installs SL probabilities. Leaf evaluation combines the value network and rollout, and backup incorporates the evidence into traversed edges. Fan uses asynchronous CPU search and GPU network evaluation, with separate statistics for neural evaluations and rollouts. A temporary pattern-based tree policy supplies guidance pending deep-policy expansion. The final move is the most visited root action (Silver et al., 2016, pp. 486–487, Methods section "Search algorithm").

The ablations connect integration to strength. The variant retaining MCTS and the value network, but removing rollouts, exceeds every other program tested. Mixed evaluation at λ = 0.5 performs best, winning at least 95% against alternative AlphaGo variants. "Value-only" thus describes leaf evaluation within MCTS. These results support complementary evaluation in Fan under the study's configuration (Silver et al., 2016, p. 488, Figure 4b).

At approximately five seconds per move, single-machine AlphaGo wins 494 of 495 games against the comparison programs; the distributed system wins 77% against the single-machine version. The latter uses 1,202 CPUs and 176 GPUs, compared with 48 CPUs and eight GPUs for the single machine. Increased search resources thus provide another measured contribution alongside learning, although accelerator counts are not an architecture-neutral measure of computation (Silver et al., 2016, pp. 487–488).

The original achievement consequently rests on coordinated changes to effective breadth, effective depth and affordable evaluation throughput. Its ablations support the combined system more strongly than any single-component narrative. They also leave a further possibility: if search is stronger than the initial policy, its recommendations could become training targets themselves. AlphaGo Lee partially moves in this direction for value learning, and Zero makes that feedback the organising principle of both outputs (Silver et al., 2016, pp. 484–488; Silver et al., 2017, pp. 3, 21).

# 4 AlphaGo Zero: search as a policy-improvement operator

## 4.1 Dual-head residual network and minimal input

Zero replaces the separate models with one shared residual tower and two heads. Its 17 binary planes contain eight successive board positions for each player's stones and one colour-to-play plane. History distinguishes repetition-relevant positions, while colour matters because komi compensates White. The simulator enforces legality and retains the full rule state beyond this compact observation. Handcrafted tactical features are removed (Silver et al., 2017, pp. 22–23, 27).

The initial convolution and the residual blocks use 256 filters. Equation 5 isolates the skip-addition principle: a block adds a learned residual transformation to its input. Zero's implemented block also includes normalisation and rectified activations. The published "20-block" and "40-block" labels correspond precisely to an initial convolutional block followed by 19 or 39 residual blocks, respectively. Counting these as individual convolutional layers would misstate depth, since each residual block contains two convolutions (Silver et al., 2017, pp. 27–29).

$$y = x + F(x) \quad (5)$$

The policy head maps the shared representation to 362 logits, one for each intersection and pass; the value head maps it through a 256-unit hidden layer to a scalar tanh output. Expressing the signed expected outcome as a win percentage requires transformation to the probability scale. These outputs share internal features but retain distinct targets and operational roles (Silver et al., 2017, pp. 6, 28–29).

The smaller configuration generated 4.9 million self-play games in three days; the larger generated 29 million in 40 days. These are separate runs with different capacities and training durations. Sharing and residual depth therefore require their own evidence rather than attribution from the final headline score. Section 5.2 examines the fixed-data architecture ablation; first, the training loop explains what the shared network learns (Silver et al., 2017, pp. 6, 8–10, 12).

## 4.2 Self-play loop and training targets

For each position, the network predicts (p, v), MCTS generates a search distribution π, and completed self-play supplies outcome z. The tuple (s, π, z) enters the replay data. Equation 6 combines value regression, cross-entropy against the search policy and parameter regularisation. The originality lies in the source of those targets: search-guided experience supplies both outputs, starting from random parameters rather than expert labels (Silver et al., 2017, pp. 5–6).

$$(p, v) = f_\theta(s) \quad L = (z - v)^2 - \Sigma_a \pi_a \log p_a + c\|\theta\|^2 \quad (6)$$

Silver et al. (2017, pp. 3, 19) explicitly interpret MCTS as a policy-improvement operator and outcomes of search-guided self-play as policy evaluation. Fitting the network projects both back into a

reusable function approximator. The improved network then guides stronger subsequent searches, closing the loop shown in Figure 1. Cross-entropy is the fitting rule; reinforcement learning describes the interaction that generates its targets. These descriptions concern different levels of the same process.

Expert Iteration provides an independent conceptual parallel. Anthony et al. (2017, pp. 1–3) distinguish a planning expert from an apprentice that generalises its decisions, then use the apprentice to strengthen planning. Their experiments concern Hex, so they support the learning–planning decomposition rather than replicate Zero's Go results. In both systems, expensive local improvement is distilled into fast predictions that can guide later computation.

Later analysis sharpens the operator interpretation. Grill et al. (2020, pp. 3770–3772) relate AlphaZero's empirical visit distribution to an approximate solution of a regularised policy-optimisation problem. Their modified visit accounting establishes an interpretation rather than a verbatim transcription of Zero's implementation. Conversely, Danihelka et al. (2022, conference abstract) identify failure to improve when root actions remain unvisited, motivating sampling without replacement. This abstract-level result qualifies automatic-improvement claims; detailed numerical comparisons remain outside the analysis.

The loop's evidence quality therefore depends on search coverage, value accuracy and the distribution of self-generated positions. Zero's successful iteration is demonstrated empirically rather than guaranteed at every gradient step. This directs attention to the specific exploration and checkpoint-selection mechanisms that control which games supply the next targets (Silver et al., 2017, pp. 23–26; Grill et al., 2020, pp. 3769–3772).

## 4.3 Search without rollouts and exploration mechanisms

Zero's search terminates a simulation at a newly evaluated leaf: one network call supplies both priors and value. Expansion and evaluation are coupled, followed by backup through the selected path. Rollouts disappear, but repeated model-based lookahead remains. Compared with Fan, every leaf is expanded, each thread waits for its neural evaluation, and the temporary tree policy is removed. The larger run additionally uses a transposition table (Silver et al., 2017, pp. 25–27).

Selection maximises Q + U, with the PUCT exploration term in Equation 7. P is the policy prior, N the visit count and $c_{puct}$ an exploration coefficient. A high prior attracts early visits; the denominator reduces the bonus as an action is sampled. Fan already uses the same structural form, although its asynchronous implementation maintains rollout and value counts separately. The transition to Zero is thus continuous in selection, while changing evaluation and target generation (Silver et al., 2016, Methods section "Search algorithm"; Silver et al., 2017, p. 26).

$$U(s,a) = c_{puct}\, P(s,a)\, \frac{\sqrt{\Sigma_b\, N(s,b)}}{1 + N(s,a)} \quad (7)$$

Self-play uses 1,600 simulations per move, approximately 0.4 seconds in the reported setup. Moves are sampled proportionally to visit counts for the first 30 moves, with temperature $\tau = 1$, and thereafter the most visited action is selected as $\tau$ approaches zero. Root priors mix 75% network probabilities with 25% Dirichlet noise, using concentration 0.03. These settings diversify complete games as well as allocating exploration within individual searches (Silver et al., 2017, p. 24).

A candidate checkpoint must win more than 55% over 400 evaluation games before replacing the incumbent data-generating player. The threshold reduces noisy replacement, but a finite match against one incumbent remains a limited test of improvement. Together, exploration and gating explain the concrete implementation of the feedback loop. Its comparative significance now depends on separating successive versions and weighing the available controls (Silver et al., 2017, pp. 23–24, 30).

# 5 Comparison and critical evaluation

## 5.1 From Fan via Lee and Master to Zero

The evolution gradually integrates search-guided self-play into learning. Fan's value targets come from RL-policy games; Lee's come from AlphaGo's own search-guided games, iterated several times. Lee also enlarges its policy and value trunks to 12 convolutional layers with 256 planes and changes inference hardware. Master already shares Zero's network architecture, RL algorithm and MCTS, but retains expert initialisation, handcrafted features and rollouts (Silver et al., 2017, pp. 21–22).

**Tab. 2 Evolution from AlphaGo Fan to AlphaGo Zero**

| Dimension | Fan | Lee | Master | Zero |
|---|---|---|---|---|
| Policy data | Human SL → RL-policy self-play | Human initialisation → self-play | Human SL initialisation → search-guided RL | Search-guided self-play from random weights |
| Value data | RL-policy game outcomes | Search-guided games; iterated | Zero-style RL pipeline | Search-guided outcomes |
| Input features | Board + handcrafted tactics | Handcrafted features retained | Lee-style handcrafted features | 17 history/colour planes |
| Rollouts | Fast pattern policy | Retained | Retained | Removed |
| Architecture | Separate convolutional policy/value | Separate 12-convolution-layer trunks; 256 planes | Same shared residual architecture as Zero | Shared residual tower; 20/40 blocks |
| Hardware for play | 176 GPUs (distributed) | 48 TPUs (distributed) | 4 TPUs, one machine | 4 TPUs, one machine |
| Training resources | Phase-specific; no common total | No common total reported | No common total reported | Optimisation: 64 GPU workers + 19 CPU servers; total self-play resources unspecified |
| Training duration | No single phase-complete duration | Several months | Not reported in comparison | 3 days: 4.9 M games; 40 days: 29 M |
| Elo, 5 s/move | 3,144 | 3,739 | 4,858 | 5,185 (40-day run) |
| Key match | 5–0 vs Fan Hui | 4–1 vs Lee Sedol | 60–0 vs top professionals online | 3-day: 100–0 vs Lee; 40-day: 89–11 vs Master |

Source: Own representation based on Silver et al. (2016, pp. 487–488, Methods section "Evaluation") and Silver et al. (2017: Fan/Lee, pp. 8, 12, 21; Master, pp. 12, 22; Zero, pp. 6, 8, 10, 12, 23–24, 27–29). Reporting limits: Tian et al. (2019, pp. 6246–6247). M = million; hardware counts refer to the specified phase, not total compute.

Table 2 separates competitive results from training and inference conditions. The Elo ladder belongs to one five-second-per-move tournament, while the 100–0 and 89–11 head-to-head results use two-hour controls. Fan's 5–0 uses a different human-match protocol; Master's 60–0 was an online series. These outcomes jointly establish progression, but their raw scores do not provide interchangeable effect sizes (Silver et al., 2016, Methods section "Evaluation"; Silver et al., 2017, pp. 8, 12, 30).

Master is especially informative because architecture and broad algorithms are already aligned with Zero. Zero's advantage over Master can consequently be interpreted as evidence about the remaining design bundle, rather than evidence that residual depth alone caused the gain. Yet expert initialisation, input features and rollouts all change together. The comparison narrows attribution without identifying an individual effect for each change (Silver et al., 2017, pp. 12, 22).

Three days and 40 days describe Zero's published runs, whereas Lee was trained over several months. Incomplete timing and resource reports leave phase-complete efficiency comparisons unresolved. The table marks those gaps explicitly. The next section accordingly gives greatest weight

to comparisons holding more components fixed (Silver et al., 2017, pp. 6, 8, 10, 21–24; Tian et al., 2019, pp. 6246–6247).

## 5.2 Attributing the performance gain

The fixed-data experiment provides the clearest architectural evidence. Four combinations of separate or dual outputs and plain convolutional or residual towers are trained on the same Zero-generated data and used within Zero's search. Residual architecture adds over 600 Elo; combining policy and value adds approximately another 600. Sharing slightly reduces human-move prediction accuracy while improving value error and playing strength. The inference is substantial joint architectural benefit under this dataset and search configuration, not a universally additive 1,200-point contribution to every version comparison (Silver et al., 2017, pp. 8–9, 29–30).

The target-source comparison holds architecture constant. The human-trained network predicts human moves better and initially plays more strongly, but the self-play player defeats it within 24 hours. This separates imitation accuracy from competitive improvement and supports the value of search-generated experience. It is nevertheless a partial control: the supervised comparator uses one-hot human move targets and downweights the value-loss term to 0.01, whereas self-play uses equally weighted losses. Experience volume and encountered positions differ too (Silver et al., 2017, pp. 8, 24–25).

Master supplies a second, less controlled comparison. Zero's final rating is 5,185 against Master's 4,858, and it wins their two-hour match 89–11. Since their architecture, RL algorithm and MCTS are shared, this 327-point rating difference supports the effectiveness of Zero's remaining design bundle. Assigning it entirely to removal of human data would confound that change with minimal features and rollout removal. Lee's earlier use of search-generated value targets further weakens a simple before-and-after story (Silver et al., 2017, pp. 12, 21–22).

The strongest evidence for decision-time planning compares the final raw network with the complete Zero agent: 3,055 versus 5,185 Elo, a 2,130-point difference on the same tournament scale. It measures the benefit of search on top of an already search-trained network, not the separate benefit of using search during training. Nor is an Elo difference a linear multiple of intelligence or efficiency. The scale is anchored through programme and human references, and the authors explicitly correct self-play bias (Silver et al., 2017, pp. 12–13, 30).

Taken together, these results support a joint explanation. Search-generated targets are the decisive conceptual change in the training loop; residual architecture and shared representation make large, quantitatively demonstrated contributions; inference-time search adds substantial further strength. The remaining attribution problem concerns interactions among features, rollouts, initialisation and resource allocation. A stronger future design would factorially vary these components under equal architecture, search budget and training compute, rather than repeat the already reported same-architecture comparison (Silver et al., 2017, pp. 8–12, 22, 29–30).

## 5.3 Limits: human knowledge, compute, robustness, reproducibility and transfer

The claim of learning without human knowledge needs an operational definition. Zero removes expert demonstrations and handcrafted tactical inputs, while retaining legal-move generation, Tromp–Taylor training scores, board symmetries, colour-relative representation and search parameters tuned by Gaussian-process optimisation. Marcus (2018, pp. 7–9) challenges literal blank-slate interpretations on these grounds, although his preprint is an interpretative critique. Subsequent AlphaZero removes symmetry augmentation and incumbent gating, showing that some design choices were dispensable, while retaining an engineered learning–search architecture (Silver et al., 2017, pp. 22–23; Silver et al., 2018, pp. 3–5).

Efficiency requires phase-specific accounting. Zero and Master play on four TPUs, versus Fan's 176 GPUs and Lee's 48 TPUs, yet Zero's network optimisation uses 64 GPU workers and 19 CPU parameter servers. Those figures exclude a complete count of self-play generation resources. Tian et al. (2019, pp. 6246–6247) independently identify this reporting gap. Accelerator generations, utilisation, training duration and energy are not normalised, so the evidence establishes a smaller playing deployment, not a verified total-cost multiplier (Silver et al., 2017, pp. 12, 23).

Independent reimplementation strengthens the credibility of the approach while exposing practical variation. ELF OpenGo publishes code, models and self-play data; its professional match record is 20–0. Importantly, that record belongs to a prototype trained with a changing hybrid process, played at approximately 80,000 search simulations per move. Its fully documented main run is distinct. This supports reproducibility of superhuman performance in the algorithm family, rather than exact reproduction of every DeepMind ablation or of the prototype's training history (Tian et al., 2019, pp. 6244, 6247).

Competitive strength also leaves exploitable blind spots. Wang et al. (2023, pp. 35655–35658) train adversaries that win more than 97% against KataGo at superhuman settings by provoking blunders, despite losing to amateur humans. This tests a successor rather than the original Zero, under a targeted adversarial threat model. It nonetheless shows why ordinary tournament Elo can miss non-transitive weaknesses. Robustness evaluation should therefore include diverse opponents and deliberately unusual positions alongside average-case matches.

Finally, Go's exact simulator is a favourable condition, not a universal prerequisite for planning. MuZero learns decision-relevant reward, policy and value predictions and matches AlphaZero's board-game performance without an externally supplied transition model (Schrittwieser et al., 2020, pp. 1–4). Real deployment still requires attention to delays, partial observability, safety constraints and limited interaction data, as documented by Dulac-Arnold et al. (2021, pp. 2420–2422). The defensible applied lesson is to validate the reliability of search-generated targets under model error and operational constraints before adopting autonomous self-training.

# 6 Conclusion

For SQ1, AlphaGo's achievement depends on a precise division of labour. The SL policy provides search priors, the RL policy generates stronger self-play behaviour, and the value network estimates that behaviour's expected outcome. A lightweight, non-deep rollout policy makes terminal continuation affordable. Distinguishing these objectives explains why the strongest policy in direct play can be less useful as a search prior, and why learning quality must be assessed in its operational role (Silver et al., 2016, pp. 485–488).

For SQ2, learned guidance makes finite lookahead more effective by concentrating breadth and reducing evaluation depth. Fan's mixed leaf evaluation wins at least 95% against the tested alternatives, and its single-machine system wins 494 of 495 games against other programs. The corresponding evidence in Zero is the 2,130-Elo gap between its final raw network and the search agent. These comparisons substantiate the importance of planning beyond a neural prediction, while remaining conditional on the evaluated budgets and opponents (Silver et al., 2016, p. 488; Silver et al., 2017, p. 12).

For SQ3 and the main question, the evidence favours joint rather than single-factor attribution. Zero closes the loop by training policy on search recommendations and value on search-guided game outcomes. The same-architecture human-data comparison supports self-play improvement, while the fixed-data ablation demonstrates substantial gains from residual architecture and shared outputs. Master provides further evidence for the remaining design bundle, but changes in features, initialisation and rollouts remain confounded. Search-generated targets are conceptually central; the available experiments cannot rank their isolated quantitative contribution above architecture (Silver et al., 2017, pp. 8–12, 22, 25).

The review is limited by purposive selection, reliance on published measurements and absence of new experiments. Version-specific pagination improves traceability; experimental validation remains dependent on the published reports. ELF OpenGo corroborates the algorithm family's capability rather than every original result, and the robustness evidence concerns KataGo. Detailed Gumbel comparisons remain outside the analysis because only the official conference abstract was accessible. These boundaries limit causal and cross-domain conclusions (Danihelka et al., 2022, conference abstract; Tian et al., 2019, pp. 6244, 6247; Wang et al., 2023, pp. 35655–35658).

Future work should build on the partial controls with resource-matched factorial ablations, including target source, input features and rollout use. Low-budget policy improvement deserves particular attention, together with adversarial opponent diversity and the reliability of learned models. This goes beyond repeating the existing imitation-versus-self-play test. For applied RL, the recommendation is to justify model fidelity, compare complete agents under a common compute budget, and evaluate safety and robustness separately from average competitive strength. The lasting contribution of Al-

phaGo is the productive coupling of generalisation with focused planning, subject to domain expertise and operational constraints (Dulac-Arnold et al., 2021, pp. 2420–2422; Grill et al., 2020, pp. 3769–3772).

# References


Anthony, T., Tian, Z., & Barber, D. (2017). Thinking fast and slow with deep learning and tree search. In I. Guyon, U. von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. V. N. Vishwanathan, & R. Garnett (Eds.), *Advances in neural information processing systems* (Vol. 30). Curran Associates. https://proceedings.neurips.cc/paper_files/paper/2017/hash/d8e1344e27a5b08cdfd5d027d9b8d6de-Abstract.html

Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning, 47*(2–3), 235–256. https://doi.org/10.1023/A:1013689704352

Browne, C. B., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., & Colton, S. (2012). A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games, 4*(1), 1–43. https://doi.org/10.1109/TCIAIG.2012.2186810

Coulom, R. (2007). Efficient selectivity and backup operators in Monte-Carlo tree search. In H. J. van den Herik, P. Ciancarini, & H. H. L. M. Donkers (Eds.), *Computers and games* (pp. 72–83). Springer. https://doi.org/10.1007/978-3-540-75538-8_7

Danihelka, I., Guez, A., Schrittwieser, J., & Silver, D. (2022). *Policy improvement by planning with Gumbel* [Conference abstract]. International Conference on Learning Representations. https://iclr.cc/virtual/2022/poster/6418

Dulac-Arnold, G., Levine, N., Mankowitz, D. J., Li, J., Paduraru, C., Gowal, S., & Hester, T. (2021). Challenges of real-world reinforcement learning: Definitions, benchmarks and analysis. *Machine Learning, 110*(9), 2419–2468. https://doi.org/10.1007/s10994-021-05961-4

Gelly, S., Kocsis, L., Schoenauer, M., Sebag, M., Silver, D., Szepesvári, C., & Teytaud, O. (2012). The grand challenge of computer Go: Monte Carlo tree search and extensions. *Communications of the ACM, 55*(3), 106–113. https://doi.org/10.1145/2093548.2093574

GitHub. (2026). *GitHub Copilot* [AI assistant]. https://docs.github.com/en/copilot/get-started/about-github-copilot

Grill, J.-B., Altché, F., Tang, Y., Hubert, T., Valko, M., Antonoglou, I., & Munos, R. (2020). Monte-Carlo tree search as regularized policy optimization. In H. Daumé III & A. Singh (Eds.), *Proceedings of the 37th International Conference on Machine Learning* (pp. 3769–3778). PMLR. https://proceedings.mlr.press/v119/grill20a.html

Kocsis, L., & Szepesvári, C. (2006). Bandit based Monte-Carlo planning. In J. Fürnkranz, T. Scheffer, & M. Spiliopoulou (Eds.), *Machine learning: ECML 2006* (pp. 282–293). Springer. https://doi.org/10.1007/11871842_29

Marcus, G. (2018). *Innateness, AlphaZero, and artificial intelligence* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1801.05667

Rosin, C. D. (n.d.). *Multi-armed bandits with episode context* [Author manuscript]. https://chris-rosin.com/isaim2010final.pdf

Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., Lillicrap, T., & Silver, D. (2020). Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature, 588*(7839), 604–609. https://doi.org/10.1038/s41586-020-03051-4. Author manuscript: https://arxiv.org/abs/1911.08265v2

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature, 529*(7587), 484–489. https://doi.org/10.1038/nature16961

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., & Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science, 362*(6419), 1140–1144. https://doi.org/10.1126/science.aar6404. Accepted manuscript: https://discovery.ucl.ac.uk/10069050/1/alphazero_preprint.pdf

Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., van den Driessche, G., Graepel, T., & Hassabis, D. (2017). Mastering the game of Go without human knowledge. *Nature, 550*(7676), 354–359. https://doi.org/10.1038/nature24270. Accepted manuscript: https://discovery.ucl.ac.uk/id/eprint/10045895/1/agz_unformatted_nature.pdf

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. http://incompleteideas.net/book/RLbook2020.pdf

Tian, Y., Ma, J., Gong, Q., Sengupta, S., Chen, Z., Pinkerton, J., & Zitnick, C. L. (2019). ELF OpenGo: An analysis and open reimplementation of AlphaZero. In K. Chaudhuri & R. Salakhutdinov (Eds.), *Proceedings of the 36th International Conference on Machine Learning* (pp. 6244–6253). PMLR. https://proceedings.mlr.press/v97/tian19a.html

Wang, T. T., Gleave, A., Tseng, T., Pelrine, K., Belrose, N., Miller, J., Dennis, M. D., Duan, Y., Pogrebniak, V., Levine, S., & Russell, S. (2023). Adversarial policies beat superhuman Go AIs. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, & J. Scarlett (Eds.), *Proceedings of the 40th International Conference on Machine Learning* (pp. 35655–35739). PMLR. https://proceedings.mlr.press/v202/wang23g.html

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning, 8*(3–4), 229–256. https://doi.org/10.1007/BF00992696